

Scholar Mind Backend

Code Explanation PDF

Generated on 2026-06-30

This backend is a FastAPI service for an academic assistant app. It accepts uploaded research documents, extracts text, sends text to an AI model for analysis or learning features, searches OpenAlex for papers, and exports references as DOCX or PDF.

1. Project Overview

- `main.py` is the FastAPI application. It defines all routes, request handling, fallback behavior, OpenAlex calls, Firestore fallbacks, and reference exports.
- `ai.py` contains the DeepSeek/OpenAI-compatible client and prompt functions for analysis, chat, flashcards, quizzes, peer review, insights, summaries, simplification, podcast scripts, and references.
- `models.py` defines Pydantic request and response models. These models validate API inputs and shape API outputs.
- `pdf.py` extracts readable text from PDF, DOCX, and legacy DOC files.
- `requirements.txt` lists runtime dependencies, including FastAPI, OpenAI client, pdfplumber, python-docx, reportlab, and multipart upload support.
- `vercel.json` configures deployment on Vercel and routes all HTTP traffic to `main.py`.
- `test_endpoints.py` is a manual smoke-test script that uses FastAPI TestClient to exercise important endpoints.

2. How A Normal Paper Analysis Works

The most important workflow is the PDF analysis flow:

```
Client uploads PDF -> main.py /api/pdf/analyze -> pdf.py extracts text -> ai.py analyze_paper asks DeepSeek for JSON -> main.py repairs/parses JSON -> Pydantic PaperAnalysis response is returned -> analysis is cached in ANALYSES.
```

- The uploaded file must end with `.pdf`.
- `extract_text_from_pdf` reads the uploaded bytes and uses pdfplumber page by page.
- The extracted text is stored in the DOCUMENTS dictionary using a generated UUID `doc_id`.
- `analyze_paper` sends the first 8000 characters to the AI model and requests a strict JSON object.
- `_repair_json` removes markdown code fences, trailing commas, and tabs from AI output before `json.loads`.
- If the AI JSON cannot be parsed, `build_fallback_analysis` creates a usable analysis from extracted text instead of crashing.

3. main.py Explained

`main.py` starts by creating the FastAPI app and enabling CORS for all origins, methods, and headers. Then it creates two in-memory stores: DOCUMENTS for extracted document text and ANALYSES for structured paper

analysis results.

Important helper functions

- `_strip_code_fences(raw_json)`: removes ```json or ``` wrappers that an AI model may add.
- `_repair_json(raw_json)`: cleans AI JSON by removing trailing commas and normalizing tabs.
- `_prepare_reference_text(text)`: shortens long papers before sending them to the reference-generation prompt, while preserving the references section when possible.
- `_extract_existing_references(text)`: looks for existing References, Bibliography, or Works Cited sections and extracts citation-like lines.
- `_parse_references_json(raw_json)`: parses AI-generated references from either a JSON array or an object containing references.
- `extract_sections_from_text(text)`: detects section headings such as Abstract, References, numbered sections, and appendices, then returns PaperSection objects.
- `build_fallback_analysis(...)`: creates a safe PaperAnalysis object if AI analysis fails.
- `get_document_context(doc_id, user_id)`: retrieves document context from memory first, then tries Firestore if a user_id is supplied.
- `_paper_analysis_from_firestore(doc_id, user_id)`: rebuilds a PaperAnalysis model from Firestore fields when local memory does not have it.

PDF endpoints

- GET / returns a simple welcome/health-check JSON response.
- POST /api/pdf/extract accepts a PDF, extracts text, stores it in DOCUMENTS, and returns doc_id plus filename.
- POST /api/pdf/analyze accepts a PDF, extracts text, asks the AI to analyze it, parses the AI response, stores the PaperAnalysis, and returns structured analysis.
- GET /api/pdf/analysis/{doc_id} returns a cached analysis or tries to rebuild it from Firestore if user_id is supplied.

Chat and learning endpoints

- POST /api/chat/beginner uses a simple explanation style.
- POST /api/chat/technical uses a more academic/technical explanation style.
- POST /api/chat/freeform answers normally using document context.
- POST /api/flashcards/generate asks AI for a JSON list of flashcards and validates each item with FlashcardItem.
- POST /api/podcast/generate turns context into a two-host podcast script.
- POST /api/summary/generate produces a short academic summary.
- POST /api/text/simplify simplifies raw text sent directly in the request.
- POST /api/quiz/generate asks AI for five MCQs. If normal JSON parsing fails, a regex fallback tries to recover usable questions.

OpenAlex paper endpoints

- GET /api/papers/search calls <https://api.openalex.org/works> with search, page, and optional filter parameters.

- GET /api/papers/trending fetches recent papers from 2024 or 2025 with citation_count greater than 5, sorted by citations.
- GET /api/papers/{paper_id} fetches one OpenAlex work by ID.
- _parse_openalex_work(work) converts raw OpenAlex JSON into the app's OpenAlexPaper model.
- _reconstruct_abstract(inverted_index) rebuilds OpenAlex abstracts from the inverted-index format OpenAlex returns.
- POST /api/papers/insights sends a paper title, abstract, and authors to AI and returns summary, findings, applications, and limitations.

Peer review and references

- POST /api/peer-review/analyze accepts PDF, DOCX, DOC, or TXT files, extracts text, and asks AI for limitations, technical flaws, and reviewer questions.
- POST /api/references/generate accepts PDF, DOCX, DOC, or TXT files, extracts text, asks AI for citations in a selected style, and falls back to references already found in the paper if the AI fails.
- POST /api/references/export/docx creates a downloadable Word document using python-docx.
- POST /api/references/export/pdf creates a downloadable references PDF using reportlab.

4. ai.py Explained

ai.py is the AI gateway. It loads DEEPSEEK_API_KEY from .env, creates an OpenAI-compatible client pointing to <https://api.deepseek.com>, and uses the deepseek-chat model.

- `_call_ai(system_prompt, user_prompt, timeout_seconds)`: shared helper that checks whether the API key exists, calls the model, retries once after failure, and returns either model text or an Error string.
- `analyze_paper(text)`: asks for a structured JSON analysis of a research paper.
- `generate_chat_response(context, message, mode)`: changes the system prompt depending on beginner, technical, or freeform mode.
- `generate_flashcards(context)`: asks for 5 to 8 flashcards as JSON.
- `generate_podcast_script(context)`: creates a short conversation between Alice and Bob.
- `generate_summary(context)`: produces a 3 to 4 sentence summary.
- `simplify_text(text)`: rewrites difficult academic text for a high-school-level reader.
- `generate_quiz_questions(context)`: asks for exactly five multiple-choice questions.
- `generate_paper_insights(title, abstract, authors)`: asks for structured insights from title and abstract.
- `generate_peer_review(text)`: asks for limitations, flaws, and reviewer questions.
- `generate_paper_references(text, style)`: asks for 8 to 12 formatted citations in APA, MLA, IEEE, Chicago, or another requested style.

5. models.py Explained

models.py contains Pydantic models. Pydantic validates incoming JSON and also lets FastAPI automatically generate API docs at /docs.

Model	Purpose
AIRequest / AIResponse	Simple prompt/response shapes, currently imported but not central.
DocumentRequest	Used by document-based generators. Contains doc_id, optional user_id, and optional context override.
ChatRequest	Used by chat endpoints. Contains doc_id, user message, optional user_id, and optional context.
SimplifyRequest	Contains text to simplify.
FlashcardItem / FlashcardResponse	Represent generated flashcards.
TextResponse	Generic response containing a text field.
PaperSection	One extracted or AI-created paper section.
PaperAnalysis	Main structured paper-analysis response.
QuizQuestion / QuizResponse	Multiple-choice quiz output.
OpenAlexPaper / OpenAlexSearchResponse	OpenAlex search/detail output.
PaperInsightsRequest / Response	Input/output for abstract-level insights.
PeerReviewResponse	Limitations, technical flaws, and reviewer questions.

Model	Purpose
ReferencesResponse / ExportReferencesRequest	Generated/exported citation data.

6. pdf.py Explained

pdf.py focuses on turning uploaded files into clean text. Academic PDFs often have columns, tables, and strange spacing, so the code tries multiple extraction strategies and selects the best-looking result.

- `PDF_TEXT_SETTINGS` stores different pdfplumber extraction configurations.
- `_clean_extracted_text` normalizes whitespace and removes excessive blank lines.
- `_text_quality_score` scores extracted text. It rewards readable length and spacing, and penalizes glued-together long words or too many single-letter tokens.
- `_words_to_lines` reconstructs lines from extracted word positions when normal text extraction is poor.
- `_extract_best_page_text` tries normal extraction, configured extraction, and word-based extraction, then chooses the best score.
- `extract_text_from_pdf` reads uploaded PDF bytes, loops through all pages, extracts best page text, and returns clean full-document text.
- `extract_text_from_docx` reads word/document.xml from the DOCX zip and joins text nodes.
- `extract_text_from_doc` handles old .doc files by first checking if it is secretly a DOCX, then scanning for readable UTF-16LE or CP1252 text runs.

7. Endpoint Summary Table

Endpoint	Method	Main job
/	GET	Health check.
/api/pdf/extract	POST	Upload PDF, extract text, return doc_id.
/api/pdf/analyze	POST	Upload PDF, extract text, AI-analyze paper, return PaperAnalysis.
/api/pdf/analysis/{doc_id}	GET	Fetch analysis from memory or Firestore fallback.
/api/chat/beginner	POST	Answer questions simply using document context.
/api/chat/technical	POST	Answer questions technically using document context.
/api/chat/freeform	POST	General document-aware chat.
/api/flashcards/generate	POST	Generate flashcards from document context.
/api/podcast/generate	POST	Generate a podcast script from document context.
/api/summary/generate	POST	Summarize document context.
/api/text/simplify	POST	Simplify provided text.
/api/quiz/generate	POST	Generate multiple-choice quiz questions.
/api/papers/search	GET	Search papers via OpenAlex.
/api/papers/trending	GET	Fetch trending recent papers via OpenAlex.
/api/papers/{paper_id}	GET	Fetch one OpenAlex paper.
/api/papers/insights	POST	Generate insights from title and abstract.
/api/peer-review/analyze	POST	Analyze uploaded paper for peer-review issues.
/api/references/generate	POST	Generate formatted references from uploaded document.
/api/references/export/docx	POST	Export references as Word file.
/api/references/export/pdf	POST	Export references as PDF file.

8. Configuration And Deployment

- DEEPSEEK_API_KEY is required for AI features. Without it, ai.py returns an Error string instead of calling the model.
- OPENALEX_API_KEY is optional. If present, it is sent to OpenAlex requests.
- .env is loaded locally through python-dotenv.
- vercel.json sets maxDuration to 60 seconds for main.py and rewrites all routes to /main.py.
- README.md explains local setup, Vercel deployment, endpoint categories, and troubleshooting.

9. Important Notes

- DOCUMENTS and ANALYSES are plain Python dictionaries. They are useful locally, but on Vercel/serverless they may disappear between invocations.
- CORS is currently open with allow_origins=[*]. This is convenient during development, but production apps usually restrict origins.
- AI output is never perfectly reliable, so the project has JSON repair and regex fallback paths.

- Some Firebase project/API-key fallback values are embedded in main.py. For production, sensitive configuration should usually come from environment variables or server configuration.
- File extension checks are used for uploads. For stronger production safety, also validate MIME type, file size, and content.
- test_endpoints.py is a smoke test, but most AI endpoints require a configured API key to succeed fully.

10. Quick Mental Model

Think of this backend as four layers: extraction, AI prompting, API orchestration, and typed responses. pdf.py performs extraction, ai.py performs model calls, main.py connects requests to those helpers and handles fallbacks, and models.py defines exactly what the app receives and returns.